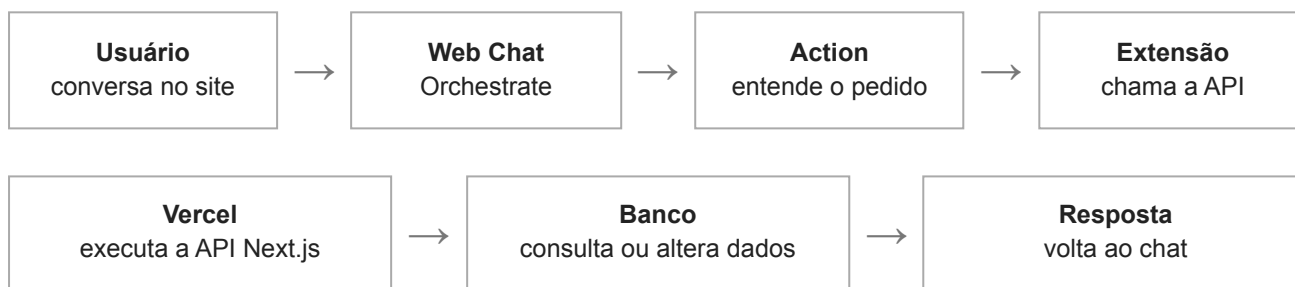


IBM watsonx Orchestrate no ReUse

Passo a passo para ligar um assistente ao projeto, chamar APIs próprias e publicar o Web Chat em uma aplicação hospedada na Vercel.

Nome correto: IBM watsonx Orchestrate. O ReUse foi construído com os recursos de **AI Assistant Builder**, Actions, extensão OpenAPI e Web Chat. Em contas antigas, a interface pode ainda mostrar “watsonx Assistant”; em contas migradas, esses recursos aparecem dentro do Orchestrate.

Como a solução funciona



O que este guia cobre

- estrutura mínima do projeto e das APIs;
- deploy inicial na Vercel e variáveis de ambiente;
- criação da extensão com OpenAPI;
- Actions de consulta e alteração com confirmação;
- Web Chat, segurança por JWT e publicação;
- testes e diagnóstico dos erros mais comuns.

Importante: este material explica a arquitetura usada no ReUse. Ele não contém chaves reais. Cada pessoa deve criar os próprios segredos e substituir domínio, rotas e nomes pelas informações do próprio projeto.

1. Preparar o projeto

Pré-requisitos

- conta IBM Cloud com acesso ao watsonx Orchestrate ou ao AI Assistant Builder;
- repositório GitHub com uma aplicação web;
- projeto criado na Vercel;
- APIs públicas por HTTPS para as tarefas do assistente;
- OpenAPI 3.0 descrevendo essas APIs.

Arquivos usados no ReUse

Arquivo	Função
components/WatsonAssistantChat.tsx	carrega o Web Chat e fornece o token de identidade;
app/api/assistant/session/route.ts	cria a identidade segura do usuário para o chat;
app/api/assistant/actions/items/*	APIs que consultam, pausam e reativam anúncios;
watson/reuse-assistant-extension.openapi.json	contrato importado como extensão;
watson/actions/action-catalog.json	frases, respostas e fluxo das Actions.

Definir primeiro as tarefas

No ReUse foram escolhidas três operações pequenas e verificáveis:

1. **consultar meus anúncios** — somente leitura;
2. **pausar anúncios disponíveis** — alteração com confirmação;
3. **reativar anúncios pausados** — alteração com confirmação.

Para um trabalho acadêmico, comece com uma operação de consulta. Só depois adicione operações que alteram dados. Isso reduz a quantidade de problemas que precisam ser investigados ao mesmo tempo.

Regra indispensável

A API não deve aceitar um identificador de usuário escolhido pelo chatbot. No ReUse, a conta é identificada pelo token emitido a partir da sessão já autenticada no site.

2. Criar as APIs e o OpenAPI

Rotas do ReUse

```
GET /api/assistant/actions/items/summary
POST /api/assistant/actions/items/pause
POST /api/assistant/actions/items/reactivate
```

Cada rota valida dois elementos:

- X-ReUse-Extension-Key: prova que a chamada veio da extensão configurada;
- X-ReUse-Action-Token: identifica a sessão e as permissões do usuário.

Estrutura essencial do OpenAPI

```
{
  "openapi": "3.0.3",
  "servers": [{ "url": "https://SEU-PROJETO.vercel.app" }],
  "security": [{ "ExtensionApiKey": [] }],
  "paths": {
    "/api/assistant/actions/items/summary": {
      "get": {
        "operationId": "getMyItemsSummary",
        "parameters": [{
          "name": "X-ReUse-Action-Token",
          "in": "header",
          "required": true
        }]
      }
    }
  }
}
```

O que precisa ser adaptado

- 1 Troque o domínio em `servers.url` pelo domínio de produção da Vercel.
- 2 Use um `operationId` único para cada tarefa.
- 3 Descreva parâmetros, corpo, respostas de sucesso e erros.
- 4 Confirme que todas as rotas respondem por HTTPS sem depender de `localhost`.
- 5 Não coloque segredo, token ou senha dentro do JSON OpenAPI.

Se o OpenAPI continuar apontando para `localhost`, para um domínio antigo ou para uma rota inexistente, a extensão será criada, mas a Action falhará ao executar.

3. Fazer o primeiro deploy na Vercel

- 1 **Importe o repositório.** Na Vercel, selecione *Add New* → *Project*, escolha o repositório do GitHub e confirme o framework detectado.
- 2 **Configure o banco e a aplicação.** Adicione primeiro as variáveis comuns do projeto, como `DATABASE_URL` e `AUTH_SECRET`.
- 3 **Faça um deploy inicial.** Esse deploy gera o endereço HTTPS que será usado em `servers.url` no OpenAPI.
- 4 **Atualize o OpenAPI.** Substitua o domínio provisório pelo endereço real da aplicação.
- 5 **Envie a alteração ao GitHub.** A Vercel criará um novo deployment automaticamente.

Variáveis utilizadas pelo ReUse

```
ASSISTANT_ACTION_SECRET="..."
ASSISTANT_EXTENSION_API_KEY="..."
ASSISTANT_WEB_CHAT_PRIVATE_KEY_BASE64="..."
ASSISTANT_WEB_CHAT_IBM_PUBLIC_KEY_BASE64="..."
NEXT_PUBLIC_IBM_ASSISTANT_INTEGRATION_ID="..."
NEXT_PUBLIC_IBM_ASSISTANT_REGION="..."
NEXT_PUBLIC_IBM_ASSISTANT_SERVICE_INSTANCE_ID="..."
NEXT_PUBLIC_IBM_ASSISTANT_WEB_CHAT_VERSION="latest"
```

Onde cadastrar

Abra **Project** → **Settings** → **Environment Variables**. Cadastre os valores em **Production**. Se também quiser testar em previews, marque o ambiente **Preview**.

Alterar uma variável não modifica deployments antigos. Depois de salvar, abra **Deployments**, localize o deployment mais recente e use **Redeploy**.

Regra para variáveis públicas

Somente os identificadores usados pelo navegador recebem o prefixo `NEXT_PUBLIC_`. Segredos, chaves privadas e tokens nunca devem usar esse prefixo.

4. Gerar segredos e chaves

Segredos independentes

Crie valores diferentes para a assinatura dos tokens e para a autenticação da extensão:

```
openssl rand -hex 32 # ASSISTANT_ACTION_SECRET
openssl rand -hex 32 # ASSISTANT_EXTENSION_API_KEY
```

O valor de `ASSISTANT_EXTENSION_API_KEY` deve ser igual na Vercel e na configuração privada da extensão IBM.

Par de chaves do Web Chat

```
openssl genpkey -algorithm RSA \
  -pkeyopt rsa_keygen_bits:2048 \
  -out web-chat-private.pem

openssl rsa -pubout \
  -in web-chat-private.pem \
  -out web-chat-public.pem
```

- **chave privada:** fica somente na Vercel;
- **chave pública:** é cadastrada na aba de segurança do Web Chat.

Converter a chave privada para base64

```
# Linux / macOS
base64 < web-chat-private.pem | tr -d '\n'

# PowerShell
[Convert]::ToBase64String(
  [IO.File]::ReadAllBytes("web-chat-private.pem")
)
```

Salve o resultado em `ASSISTANT_WEB_CHAT_PRIVATE_KEY_BASE64`.

Chave pública da IBM

Quando a proteção de dados privados estiver configurada, copie a chave pública de criptografia fornecida pela IBM e salve-a em `ASSISTANT_WEB_CHAT_IBM_PUBLIC_KEY_BASE64`, no formato esperado pela implementação.

Nunca envie chaves privadas por mensagem, nunca as coloque no GitHub e nunca reutilize segredos de outro projeto. Se uma chave privada for exposta, gere um novo par.

5. Criar o assistente e importar a extensão

- 1 Abra o **watsonx Orchestrate** e entre no **AI Assistant Builder**. Em uma conta não migrada, o produto pode aparecer como **watsonx Assistant**.
- 2 Crie um assistente, escolha **Português (Brasil)** e trabalhe primeiro no ambiente de rascunho.
- 3 Abra o catálogo de integrações e procure **Extensions**.
- 4 Selecione **Build custom extension** e importe o arquivo OpenAPI.
- 5 Revise as operações encontradas. No ReUse são: `getMyItemsSummary`, `pauseMyAvailableItems` e `reactivateMyPausedItems`.
- 6 Adicione a extensão criada ao assistente. Apenas importar o arquivo não torna as operações disponíveis nas Actions.

Autenticação da extensão

Configure a API key no cabeçalho:

```
X-ReUse-Extension-Key: valor igual ao  
ASSISTANT_EXTENSION_API_KEY da Vercel
```

Token privado da sessão

Em cada operação, associe o cabeçalho `X-ReUse-Action-Token` ao valor privado recebido pelo Web Chat:

```
${system_integrations.chat.private.user_payload}  
.reuse_action_token
```

A expressão pode variar conforme a versão da interface. O ponto essencial é usar o campo privado `reuse_action_token`, e não pedir ao usuário que digite o próprio token.

Quando a importação falhar

- valide se o JSON é OpenAPI 3.0;
- confirme que os `operationId` são únicos;
- remova recursos não suportados e importe novamente;
- se alterar a especificação, algumas versões exigem excluir e recriar a extensão.

6. Criar as Actions

Action de consulta

- 1 Crie a Action “Consultar meus anúncios”.
- 2 Adicione exemplos de frases: “quais são meus anúncios?”, “quantos itens tenho?” e “mostre meus itens”.
- 3 Adicione um passo que chama `getMyItemsSummary`.
- 4 Use os campos da resposta para montar uma frase curta para o usuário.

Action que altera dados

Para pausar anúncios, o fluxo usado no ReUse é:



1. Chame `getMyItemsSummary`.
2. Se não houver item disponível, encerre com uma mensagem informativa.
3. Se houver, pergunte: “Você quer pausar todos agora?”.
4. Execute `pauseMyAvailableItems` somente após resposta positiva.
5. Mostre a mensagem devolvida pela API.

Variáveis públicas do contexto

O componente do ReUse envia informações não sensíveis, como:

```
reuse_authenticated  
reuse_token_expires_at  
reuse_user_name
```

O token que autoriza as APIs permanece no `private.user_payload`.

Não confunda “usuário autenticado no site” com “chat aberto”. A Action deve impedir operações privadas quando `reuse_authenticated` for falso.

7. Configurar e incorporar o Web Chat

- 1 Abra **Integrations** → **Web chat**.
- 2 Configure nome, saudação, cor e mensagens iniciais.
- 3 Copie `integrationID`, `region` e `serviceInstanceID`.
- 4 Cadastre os três valores nas variáveis `NEXT_PUBLIC_IBM_...` da Vercel.
- 5 Na aba **Security**, ative **Secure your web chat**.
- 6 Cole a chave pública gerada no passo 4.
- 7 Salve e faça um novo deployment na Vercel.

Configuração usada no navegador

```
window.watsonAssistantChatOptions = {  
  integrationID,  
  region,  
  serviceInstanceID,  
  clientVersion: "latest",  
  identityToken,  
  onLoad(instance) {  
    instance.updateLocale("pt-BR");  
    instance.render();  
  }  
};
```

Como o token é obtido

Antes de carregar o chat, o ReUse chama `POST /api/assistant/session`. Essa rota:

- lê a sessão do site;
- cria um token curto para as Actions;
- coloca esse token no payload privado;
- assina a identidade do Web Chat com a chave privada;
- devolve ao navegador apenas o JWT necessário para iniciar o chat.

O navegador nunca recebe a chave privada da Vercel. Ele recebe apenas um token assinado e com validade limitada.

8. Publicar e testar

Publicação

- 1 No Assistant Builder, crie uma versão do conteúdo.
- 2 Associe essa versão ao ambiente **Live**.
- 3 Confirme que os identificadores do Web Chat pertencem ao ambiente correto.
- 4 Na Vercel, faça um redeploy após a última alteração de variáveis.

Teste em ordem

1. Abra a aplicação em janela anônima e confirme que o botão do chat aparece.
2. Faça uma pergunta apenas de orientação.
3. Faça login e consulte dados da conta.
4. Teste uma alteração e recuse a confirmação.
5. Repita, confirme e verifique o resultado também fora do chat.
6. Entre com outra conta e confirme que ela não recebe os dados da primeira.

Diagnóstico rápido

Sintoma	Verificação
chat não aparece	IDs públicos ausentes; deployment antigo; componente não montado;
erro 401 na extensão	API key IBM diferente da variável na Vercel;
erro 403 ou token inválido	payload privado não mapeado; token expirado; segredo divergente;
extensão dá 404	servers.url ou caminho da rota incorreto;
funciona no Draft, não no site	versão não publicada em Live ou IDs de outro ambiente;
mudou variável e nada aconteceu	é necessário gerar um novo deployment;
funciona localmente, não na Vercel	variável cadastrada no ambiente errado ou dependência de localhost.

9. Checklist final e referências

Projeto e Vercel

- ☐ APIs funcionam por HTTPS;
- ☐ OpenAPI aponta para produção;
- ☐ segredos estão somente na Vercel;
- ☐ novo deployment foi criado.

IBM

- ☐ extensão foi importada e adicionada;
- ☐ API key está configurada;
- ☐ Actions chamam as operações;
- ☐ versão Live foi publicada.

Web Chat e segurança

- ☐ IDs públicos são do ambiente correto;
- ☐ proteção do Web Chat está ativada;
- ☐ IBM possui a chave pública do projeto;
- ☐ chave privada está somente na Vercel;
- ☐ operações privadas exigem sessão autenticada;
- ☐ alterações pedem confirmação explícita.

Referências do ReUse

github.com/lucasbuzato/ReUse

[docs/IBM-WATSON-SETUP.md](#)

[watson/reuse-assistant-extension.openapi.json](#)

Documentação oficial

[IBM – Building a custom extension](#)

[IBM – Enabling web chat security](#)

[IBM – Migrating assets from watsonx Assistant](#)

[Vercel – Managing environment variables](#)

Resumo: o Orchestrate entende a conversa; a extensão descreve as APIs; a Vercel executa as tarefas; o token liga cada chamada à sessão correta; e o Web Chat conecta tudo ao site.